

Assignment 5 – Home assignment: Predictive control of the nonlinear pendulum

Subject: Intelligent Systems in Decision Support

The goal of this assignment is to close the loop around the nonlinear pendulum from Assignment 4. In Assignment 4 you generated data and trained prediction models for the pendulum process. In this assignment you use such a model inside a model predictive controller (MPC), using the same receding-horizon idea and the same input parameterisations as in Live Assignment 5.

The assignment therefore connects the two previous topics:

- **system identification:** the controller needs a model that can predict the pendulum output several samples ahead (Live Assignment 4 and Home Assignment 4);
- **predictive control:** the controller computes an input sequence by minimising a finite-horizon tracking criterion, applies only the first input, and repeats this procedure at every sample (Live Assignment 5).

Goal of the assignment

- Control the nonlinear pendulum in the region $\varphi \in [20^\circ, 80^\circ]$ using MPC with identified model from home assignment 4.
- If there is error in the steady-state it is OK as this class is not about control details, but explain why such error and how to mitigate it.
- Evaluate the performance.

The process

The controlled process is the same pendulum as in Assignment 4, so use the Assignment 4 homework files as the reference for the process interface. The input is the applied torque $u(k)$ and the measured output is the angle $\varphi(k)$ in radians. The region in which we want to control the pendulum is

$$\varphi \in [20^\circ, 80^\circ].$$

The simulator includes nonlinear gravity, friction, measurement noise, angle quantisation and mechanical end-stops at 10° and 100° . The actuator is also part of the process: the applied torque is saturated to

$$0 \text{ Nm} \leq u(k) \leq 3 \text{ Nm}.$$

These limits are not MPC tuning parameters; they describe what the process actuator can physically apply.

For MATLAB, use the file `pendulum_process.m`. Reset it before a new run with

`pendulum_process([], 0).`

For Python, use the file `pendulum_process.py`. It implements the same process dynamics and the same measurement-noise settings directly in Python and does not require MATLAB. The actual random noise sequence may differ from MATLAB.

Provided reference signal

The file `vaja5_reference.{m,py}` generates the reference trajectory used for the homework evaluation. The reference is reproducible for a fixed random seed and has two parts:

- first, random step changes with random holding times;
- second, a smooth random signal.

All reference values stay inside the normal operating region $[20^\circ, 80^\circ]$. Use this generator in your simulations instead of manually defining your own reference.

MPC formulation

At sample k the controller receives the recent measured output $\varphi(k)$ and input history $u(k-1), \dots$. The goal is to find future input values

$$u(k), u(k+1), \dots, u(k+H-1)$$

that will make the predicted future output

$$\hat{\varphi}(k+1), \hat{\varphi}(k+2), \dots, \hat{\varphi}(k+H)$$

track the reference trajectory as closely as possible. Here H is the prediction horizon.

Instead of optimizing all $u(k), u(k+1), \dots, u(k+H-1)$, we describe the future input by a short vector of parameters

$$\boldsymbol{\theta} = [\theta_1, \theta_2, \dots, \theta_S]^T.$$

The actual process input is still denoted by u . The parameters θ_s have the same physical units as the input torque and are expanded into the full future input sequence by holding each θ_s constant over one segment:

$$u(k+j-1) = \begin{cases} \theta_1, & 1 \leq j \leq L_1, \\ \theta_2, & L_1 < j \leq L_1 + L_2, \\ \vdots & \vdots \\ \theta_S, & \sum_{i=1}^{S-1} L_i < j \leq H, \end{cases} \quad j = 1, \dots, H.$$

The finite-horizon criterion is

$$J(\boldsymbol{\theta}) = \sum_{j=1}^H (r(k+j) - \hat{\varphi}(k+j|\boldsymbol{\theta}))^2 + \lambda \sum_{s=1}^S (\theta_s - \theta_{s-1})^2, \quad \theta_0 := u(k-1), \quad (1)$$

At each sampling instant the controller:

1. builds a reference window by holding the current set-point over the prediction horizon;
2. optimises θ by minimising (1);
3. applies only the first input $u(k) = \theta_1^*$ to the real process;
4. shifts the measured input–output history and repeats the optimisation at the next sample.

This is the receding-horizon principle.

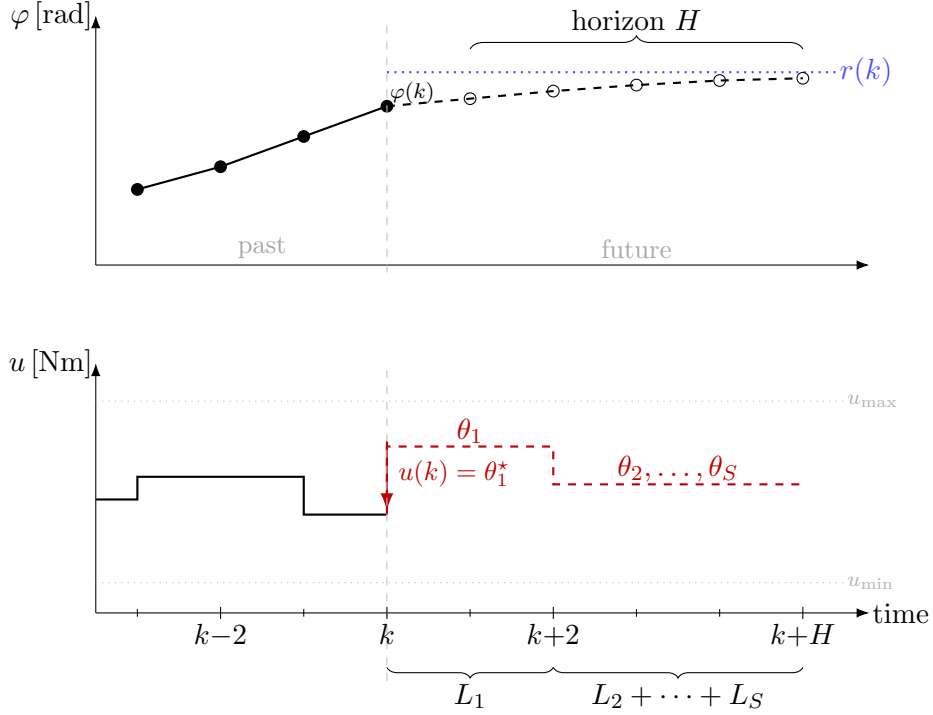


Figure 1: Receding-horizon MPC principle. *Top*: filled circles are past measured outputs φ ; open circles connected by a dashed line are values predicted by the model over the horizon H ; the dotted line is the reference $r(k)$, held constant at its current value. *Bottom*: solid steps are past applied torques; dashed steps are the optimized future input, parameterized by S values $\theta_1, \dots, \theta_S$ each held constant over a block of samples (of lengths $L_1, \dots, L_S$). Only the first input $u(k) = \theta_1^*$ is applied to the process; the horizon then shifts by one sample (*receding horizon*).

Your task

No student skeleton is provided for this homework.

Use the best prediction model you obtained in Assignment 4. You may use a linear model, a fuzzy Takagi–Sugeno model, a neural network, or another justified dynamic model, provided that it can be simulated freely over the prediction horizon.

Do not confuse the MPC parameters θ with the parameters of your prediction model. In the code, the model parameters are named separately from the optimised future-input parameters.

Reference trajectory in MPC

The controller does not get preview of future reference changes. At every sample the current value of the generated reference is held constant over the full prediction horizon. This means the controller reacts to a step only when the step happens, not before.

Implementation notes

- Use a bounded nonlinear optimiser (`fmincon` in MATLAB, SLSQP via `minimize` in Python).
- Warm-start the optimizer with the previous optimum.

What to submit

Submit a report and the code you used. The code need to run, and be well structured and commented. The report should contain:

- The short description of the implemented controller and the prediction model;
- All parts, that needed to be implemented for MPC to be implemented for the pendulum process;
- Results - what to measure and compare is part of the assignment, you can also include other results that you find interesting;
- Discussion of the results and conclusions.

Files

- `vaja5_reference.m` – MATLAB reference generator;
- `vaja5_reference.py` – Python reference generator;
- `pendulum_process.m` – MATLAB pendulum process from Assignment 4;
- `pendulum_process.py` – matching Python pendulum process;
- `example_pendulum_process.m` – MATLAB process example;
- `example_pendulum_process.py` – Python process example;